케이녹 C언어 5회차 과제

17기 최동현 (202511688)

1. 다중 포인터 (Multiple Pointer)

• 다중 포인터(Multiple Pointer)란?

: C언어에서 Pointer to Pointer라고도 불리는 **다중 포인터**는 (*포인터 변수*)의 (*메모리 주소*)를 저장하는 포인터입니다. 일반적인 **단일 포인터**(***p**)는 실제 데이터가 담긴 변수의 메모리 주소를 가리키지만, **다중 포인터**(****p**)는 실제 변수의 주소를 저장하고 있는 포인터의 주소를 저장합니다.

```
// 다중 포인터 (Multiple Pointer)

// 이중 포인터
자료형** 이중포인터명; // 이중 포인터(Double Pointer) 선언
이중포인터명 = 포인터주소값(&); // 주소값 할당

// 삼중 포인터
자료형*** 삼중포인터명;
삼중포인터명 = 이중포인터주소값(&);

// ... 이론상 무한히 가능합니다.
```

1-1. 다중 포인터 (Multiple Pointer)

• 다중 포인터(Multiple Pointer) 사용하는 이유?

: C언어에서 이러한 **다중 포인터**를 **사용하는 이유**는 여러 가지지만, **크게 두가지** 이유가 있습니다.

1) **함수 안**에서 **원본 포인터(인자 주소)**의 **가리키는 주소(방향)**를 바꿀 때.
   - 예를 들어 함수가 반환하는 **return 값**이 실행 **성공 여부만**을 **반환**한다면,
   - **매개변수**를 **이중 포인터**로 사용하여 **원본 포인터**의 **가리키는 방향**을 **새로 할당한 메모리로 연결** 할 때, 이 2중 포인터를 사용하면 좋습니다.
   - example) `int MemAlloc(int** dp) { ... }`

2) **동적**인 **2차원 배열**을 만들 때.
   - **컴파일 시** 크기가 결정되는 **스택(Stack) 영역**과 다르게 **힙(Heap) 영역**을 사용하여 **프로그램 실행 시 메모리를 할당** 받아 크기를 결정하는 **2차원 배열** 프로그램을 만들고 싶을 때, 이 이중 포인터를 사용하여 2차원 배열을 구현하면 Stack 보다 **더 유연하게 관리**하는 것이 가능해집니다.
   - example)
   - `int rows = 3, cols = 3; // (3 x 3) 2차원 행렬`
   - `int** matrix = (int**)malloc(sizeof(int*) * rows); // 행 할당`
   - `for(int i = 0; i < rows; i++) { *(matrix+i) = (int*)malloc(sizeof(int) * cols); } // 열 할당`

1-2. 다중 포인터 (Multiple Pointer)

- C언어 다중 포인터 (예시 코드화면 및 실행화면)

```c
#include <stdio.h>

int main(void) {
    int origin = 10;
    int *ptr = &origin; // origin의 메모리 주소 할당
    int **doublePtr = &ptr; // ptr의 메모리 주소 할당

    printf("원본(origin) 변수 주소 : %p\n", ptr);
    // ptr이 가지고 있는 origin 주소
    printf("포인터(ptr) 변수 주소 : %p\n", doublePtr);
    // doublePtr이 가지고 있는 ptr 주소
    printf("이중 포인터(doublePtr) 변수 주소 : %p\n\n", &doublePtr);
    // doublePtr의 주소

    printf("원본(origin) 실제 값 : %d\n", origin);
    printf("포인터(ptr)를 통한 역참조 값 : %d\n", *ptr);
    printf("이중 포인터(doublePtr)를 통한 이중 역참조 값 : %d\n\n", **doublePtr);

    **doublePtr = 100;
    printf("변경 된 원본(origin) 값 : %d\n", origin); // 100 출력

    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day5/main.c

원본(origin) 변수 주소 : 0x7ff7b8384cf8
포인터(ptr) 변수 주소 : 0x7ff7b8384cf0
이중 포인터(doublePtr) 변수 주소 : 0x7ff7b8384ce8

원본(origin) 실제 값 : 10
포인터(ptr)를 통한 역참조 값 : 10
이중 포인터(doublePtr)를 통한 이중 역참조 값 : 10

변경 된 원본(origin) 값 : 100

계속하려면 엔터 혹은 명령을 입력하십시오
```

1-3. 다중 포인터 (주의 사항!)

```c
#include <stdio.h>

int isAdmin = 0; // 0은 일반 유저, 1은 관리자
int normalData = 10;

// 이중 포인터를 사용하여 이중역참조로 값을 변경하는데,
// 아무 조건없이 그냥 주소 받아서 값을 조작함.
void modifyData(int **targetPtr) {
    **targetPtr = 1;
}

int main() {
    int *dataPtr = &normalData;
    int **doublePtr = &dataPtr;

    dataPtr = &isAdmin;
    // 공격자가 오버플로우 등으로 dataPtr에 isAdmin 주소 값 넣었다고 가정

    modifyData(targetPtr: doublePtr); // normalData가 1이 되야하는데 isAdmin이 1이 되버림

    if (isAdmin == 1) {
        printf("권한 상승(Privilege Escalation) 성공, 관리자 권한 획득.\n");
    }

    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day5/main.c

권한 상승(Privilege Escalation) 성공, 관리자 권한 획득.

계속하려면 엔터 혹은 명령을 입력하십시오
```

2-1. 포인터 배열 (Array of Pointer)

- **포인터 배열(Array of Pointer)이란?**

  : C언어에서 **포인터 배열**은 **배열의 모든 요소**(**Index**)가 **포인터(메모리 주소)**로 구성된 **자료 구조**입니다.
  해당 이 포인터 배열은 데이터가 존재하는 **메모리**의 **주소 값**들이 **연속적**으로 **배치**됩니다.

```
// 포인터 배열 (Array of Pointer) 선언 및 초기화

자료형* 배열명[배열크기(INDEX)] = { 주소값(&), 주소값(&), ... };

// 예시 코드 ([.rodata]문자열 리터럴 주소값 할당)
char* strArr[3] = { "Hello", "K-knock", "World!" };
```

2-1. 포인터 배열 (Array of Pointer)

- C언어 포인터 배열 (예시 코드화면 및 실행화면)

```c
#include <stdio.h>

int main(void) {
    char* stringArray[3] = { // [.rodata] 문자열 리터럴 시작 주소
        [0]="K-knock",
        [1]="Security",
        [2]="Hacking"
    };

    for(int i = 0; i < sizeof(stringArray) / sizeof(stringArray[0]); i++) {
        printf("stringArray[%d] 주소값 : %p, 문자열 : %s\n", i, stringArray[i], stringArray[i]);
    }

    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day5/main.c

stringArray[0] 주소값 : 0x10935356a, 문자열 : K-knock
stringArray[1] 주소값 : 0x109353572, 문자열 : Security
stringArray[2] 주소값 : 0x10935357b, 문자열 : Hacking

계속하려면 엔터 혹은 명령을 입력하십시오
```

2-2. 배열 포인터 (Pointer to an Array)

- **배열 포인터(Pointer to an Array)란?**

  : C언어에서 **배열 포인터**는 **배열의 특정 요소** 하나를 가리키는 것이 아니라, **특정 크기를 가진 배열 전체**를 **하나의 단위**로 가리키는 단일 포인터입니다.

```
// 배열 포인터 (Pointer to an Array) 선언 및 초기화
자료형 (*배열포인터명)[가리킬_배열의_크기]; // 반드시 포인터명 주위 괄호 포함해야 함

// 배열 포인터 초기화
자료형 배열명[크기]; // 배열 선언
배열포인터명 = &배열명; // 배열 전체의 주소를 할당
```

2-2. 배열 포인터 (Pointer to an Array)

- C언어 배열 포인터 (예시 코드화면 및 실행화면)

```c
#include <stdio.h>

int main() {
    int arr[3] = {[0]=10, [1]=20, [2]=30};

    int (*ptr_to_arr)[3] = &arr;
    // 정수형 3개짜리 배열을 가리킬 수 있는 배열 포인터

    printf("배열의 첫 번째 요소 주소: %p\n", arr);
    printf("배열 포인터가 가진 주소: %p\n", ptr_to_arr);

    // 일반 포인터와 배열 포인터 비교
    int *normal_ptr = arr;

    printf("일반 포인터 (+1)이동 주소: %p ", normal_ptr + 1);
    printf("[%lu바이트 증가]\n", (long)(normal_ptr + 1) - (long)(arr));
    printf("배열 포인터 (+1)이동 주소: %p ", ptr_to_arr + 1);
    printf("[%lu바이트 증가]\n", (long)(ptr_to_arr + 1) - (long)(arr));

    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day5/main.c

배열의 첫 번째 요소 주소: 0x7ff7bc9edcec
배열 포인터가 가진 주소: 0x7ff7bc9edcec
일반 포인터 (+1)이동 주소: 0x7ff7bc9edcf0 [4바이트 증가]
배열 포인터 (+1)이동 주소: 0x7ff7bc9edcf8 [12바이트 증가]

계속하려면 엔터 혹은 명령을 입력하십시오
```

3. 실습 과제 (두 문자열 같은지 비교)

요구사항

1. 사용자로부터 세 가지 입력을 받으세요:
   - 원본 문자열
   - 대체할 문자열(찾을 문자열)
   - 새로운 문자열(바꿀 문자열)
2. 다음 기능을 가진 `equal` 함수를 구현하세요:

```c
/**
 * @brief 두 문자열이 같은지 비교하는 재귀 함수
 *
 * @param str1 첫 번째 문자열
 * @param str2 두 번째 문자열
 * @return int 같으면 1, 다르면 0 반환
 */
int equal(char str1[], char str2[]);
```

제약 조건

- 문자열의 최대 길이는 100자로 제한합니다.
- 사용자 입력은 표준 입력(stdin)을 통해 받으세요.
- `string.h` 헤더에서는 오직 `strcpy` 와 `strlen` 함수만 사용 가능합니다. 다른 문자열 관련 함수 (`strcmp` , `strcat` , `strstr` 등)는 사용하지 마세요.
- 포인터 연산을 적절히 활용하세요.
- `equal` 함수는 반드시 재귀 함수로 구현해야 합니다.

3. 실습 과제 (두 문자열 같은지 비교)

```c
#include <stdio.h>

int equal(char str1[], char str2[]) { // 두 문자열 시작주소를 인자로 받음.
    if (*str1 != *str2) return 0;
    // 두 문자열 시작 주소의 문자 값을 하나씩 비교하여 다르다면 0 반환
    if (*str1 == '\0') return 1;
    // NULL 값에 다달았다면 문자열이 전부 일치했음으로 1 반환
    return equal(str1: str1 + 1, str2: str2 + 1);
    // 재귀를 통해 문자열의 첫 문자부터 끝까지 비교 위해 재귀하여 반복
}

int main(void) {
    char str1[101] = {[0]='\0'}; // NULL 문자 공간 위해 +1
    char str2[101] = {[0]='\0'};

    printf("첫 번째 문자열 입력 : ");
    fgets(str1, size: sizeof(str1), stdin);
    printf("두 번째 문자열 입력 : ");
    fgets(str2, size: sizeof(str2), stdin);

    if (equal(str1, str2)) {
        printf("문자열이 서로 일치합니다!\n");
    } else {
        printf("문자열이 서로 일치하지 않습니다.\n");
    }

    return 0;
}
```

```
~/Desktop/CHOI  vim ./KKnock/Day5/main.c

첫 번째 문자열 입력 : test1
두 번째 문자열 입력 : test2
문자열이 서로 일치하지 않습니다.

계속하려면 엔터 혹은 명령을 입력하십시오
첫 번째 문자열 입력 : Hello K-knock!
두 번째 문자열 입력 : Hello K-knock!
문자열이 서로 일치합니다!

계속하려면 엔터 혹은 명령을 입력하십시오
```

감사합니다

17기 최동현